แบบฝึกหัดปฏิบัติการคาบที่ 7: Polymorphism

1. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสืบทอดคลาสจากตัวอย่างต่อไปนี้

```
class A{
   void f(){
      System.out.println("A");
   }
}
class B extends A{
   void f(){
      System.out.println("B");
   }
}
class Test38{
   static void t(A a){
      a.f();
   }
   public static void main(String[] args){
      t(new A());
      t(new B());
   }
}
```

ให้อธิบายผลลัพธ์ของโปรแกรมพร้อมอธิบายเหตุใดจึงได้คำตอบดังกล่าว

```
A
B
```

เมธอด f() ถูก override ในคลาส B เมื่อเรียก t(new A()) จะแสดง A และเมื่อเรียก t(new B()) จะแสดง B เพราะ Java ใช้ Dynamic Binding ในการเลือกเมธอดตามชนิดของ object จริงขณะทำงาน

```
class A{
   int x=1;
   void inc(){x++;}
   int get(){return x;}
}
class B extends A{
   int x=2;
   void inc(){x++;}
   int get(){return x;}
}
class Test{
  public static void main(String[] args){
      B b =new B();
      A a= b;
      a.inc();
      b.inc();
      System.out.println(a.x+","+b.x);
      System.out.println(a.get()+","+b.get());
   }
}
```

ให้อธิบายถึงส่วนที่มีการทำงานเป็น Static binding และ Dynamic binding

```
1,4
4,4
```

การเข้าถึงตัวแปร x เป็น Static Binding โดยพิจารณาจากชนิดของตัวแปรอ้างอิง ดังนั้น a.x อ้างถึง x ของคลาส A และ b.x อ้างถึง x ของคลาส B ส่วนการเรียกเมธอด inc() และ get() เป็น Dynamic Binding โดยเลือกเมธอดตามชนิดของ object จริง ซึ่งเป็น B ทำให้ a.inc() และ b.inc() เพิ่มค่า x ของคลาส B ทั้งคู่

```
public class Test{
   public static void main(String[] args){
      Object fruit = new Fruit();
      Object apple= new (Apple)fruit;
   }
}
class Apple extends Fruit{ }
class Fruit{}
```

จงอธิบายส่วนที่ผิดของโปรแกรม

เขียนการสร้าง object ไม่ถูกต้อง และไม่สามารถใช้ new ตามด้วยการ cast ได้

2. จงอธิบายผลลัพธ์ของ Output ต่อไปนี้

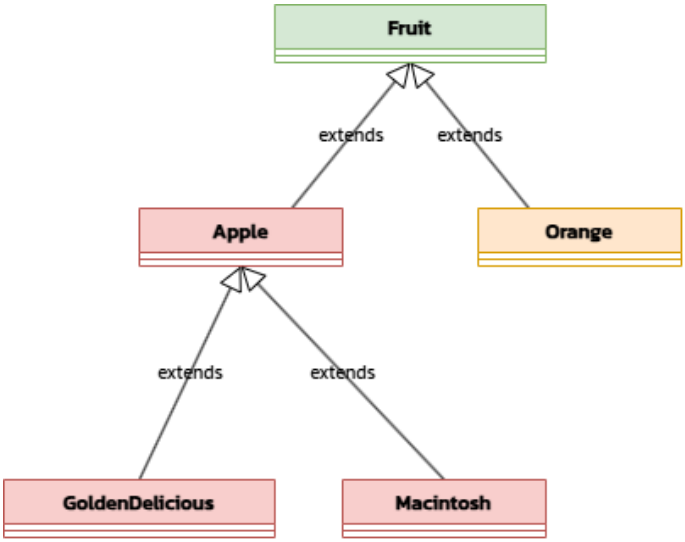
โปรแกรม	ผลลัพธ์ / คำอธิบาย

`public class Test {`
`  public static void main(String[] args) {`
`  Object a1 = new A();`
`  Object a2 = new Object();`
`  System.out.println(a1);`
`  System.out.println(a2);`
`  }`
`}`
`class A {`
`  int x;`
`  public String toString() {`
`  return "A's x is " + x;`
`  }`
`}`
	`A's x is 0`
	`java.lang.Object@8bcc55f`
	
	System.out.println(a1) จะเรียกใช้เมธอด toString() ของคลาส A จึงแสดงผลตามบรรทัดแรก เนื่องจากตัวแปร x มีค่าเริ่มต้นเป็น 0 ส่วน a2 เป็น object ของคลาส Object จึงใช้เมธอด toString() ของคลาส Object

`public class Test {`
`  public static void main(String[] args) {`
`  Object a1 = new A();`
`  Object a2 = new A();`
`  System.out.println(a1.equals(a2));`
`  }`
`}`
`class A {`
`  int x;`
`  public boolean equals(A a) {`
`  return this.x == a.x;`
`  }`
`}`
	`false`
	
	เพราะ a1 และ a2 มีชนิดเป็น Object ทำให้เรียกใช้เมธอด equals(Object) ของคลาส Object ซึ่งเปรียบเทียบว่าเป็น object เดียวกันหรือไม่ แม้ว่าทั้งสอง object จะมีค่า x เท่ากัน แต่เป็นคนละ object กัน

`public class Test {`
`  public static void main(String[] args) {`
`  A a1 = new A();`
`  A a2 = new A();`
`  System.out.println(a1.equals(a2));`
`  }`
`}`
`class A {`
`  int x;`
`  public boolean equals(A a) {`
`  return this.x == a.x;`
`  }`
`}`
	`true`
	
	เพราะ a1 และ a2 มีชนิดเป็น A จึงเรียกใช้เมธอด equals(A a) ของคลาส A ซึ่งเปรียบเทียบค่าของตัวแปร x ของทั้งสอง object โดยทั้งคู่มีค่าเริ่มต้นเป็น 0 จึงได้ผลลัพธ์เป็น true

3. กำหนดให้ Fruit, Apple, Orange, Golden Delicious Apple, และ Macintosh Apple มีโครงสร้างการสืบทอดและการประกาศตัวแปรดังนี้ ให้เขียนโปรแกรมเพื่อจำลองการทำงานของโครงสร้างของคลาสดังกล่าวพร้อมตอบคำถามต่อไปนี้



กำหนดการทำงานภายใน main() มีการสร้าง Object ดังนี้

```
Fruit fruit = new
GoldenDelicious();
Orange orange = new Orange();
```

1. Is fruit instanceof Orange?
2. Is fruit instanceof Apple?
3. Is fruit instanceof GoldenDelicious?
4. Is fruit instanceof Macintosh?
5. Is orange instanceof Orange?
6. Is orange instanceof Fruit?
7. Is orange instanceof Apple?
8. Suppose the method makeApple() is defined in the Apple class. Can fruit invoke this method? Can orange invoke this method?
9. Suppose the method makeOrangeJuice() is defined in the Orange class. Can orange invoke this method? Can fruit invoke this method?

Output จาก P03_Fruit:

```
1. Is fruit instanceof Orange? false
2. Is fruit instanceof Apple? true
3. Is fruit instanceof GoldenDelicious? true
4. Is fruit instanceof Macintosh? false
5. Is orange instanceof Orange? true
6. Is orange instanceof Fruit? true
7. Is orange instanceof Apple? false
8. Can fruit and orange invoke makeApple()?
fruit: Yes, because the actual object is a GoldenDelicious, which
inherits from Apple. orange: No, because Orange does not define or
inherit makeApple().
9. Can orange and fruit invoke makeOrangeJuice()?
orange: Yes, because Orange defines the method. fruit: No, because
the reference type Fruit does not declare makeOrangeJuice().
```

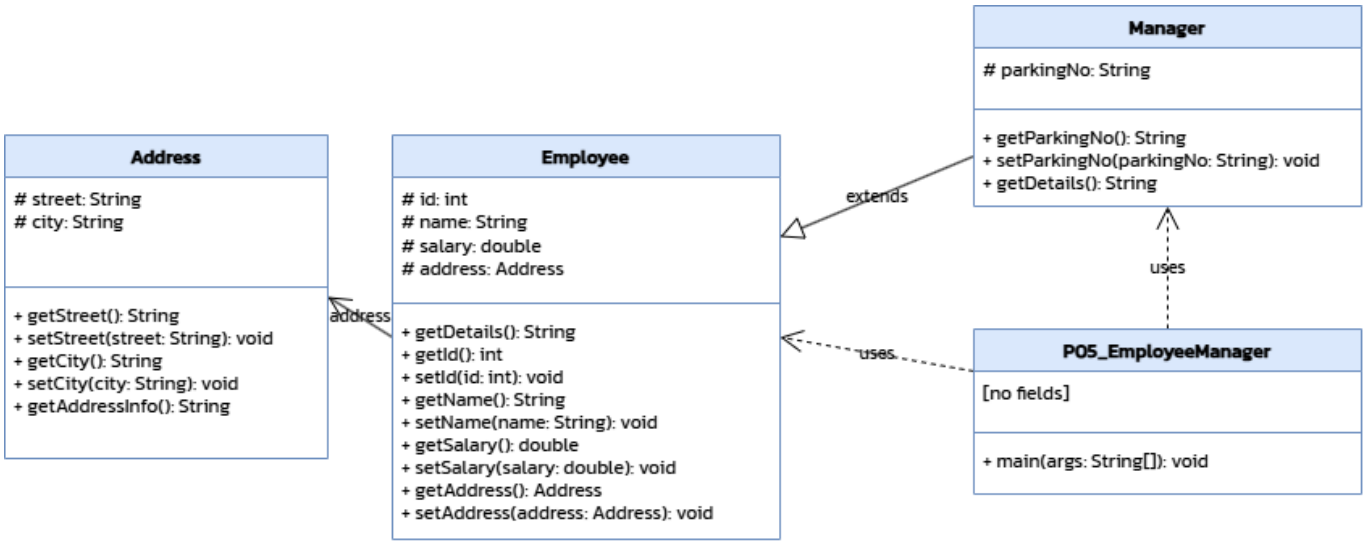
4. พิจารณาส่วนของโปรแกรมต่อไปนี้

```
1 | public class Test {
2 |     public static void main(String[] args) {
3 |         Animal x = new Tiger();
4 |
5 |         System.out.println("1. x.news is " + x.news);
6 |         System.out.println("2. ((Tiger)x).news is " + ((Tiger)x).news);
7 |         System.out.println("3. x.smile() is " + x.smile());
8 |         System.out.println("4. ((Tiger)x).smile() is " + ((Tiger)x).smile());
9 |         System.out.println("5. x.getNews() is " + x.getNews());
10|         System.out.println("6. x.getMessage() is " + x.getMessage());
11|     }
12| }
13|
14| class Animal {
15|     public String news = "Animal's news";
16|     public String message = "Animal's message";
17|
18|     public static String smile() {
19|         return "smile from Animal";
20|     }
21|
22|     public String getNews() {
23|         return news;
24|     }
25|
26|     public String getMessage() {
27|         return message;
28|     }
29| }
30|
31| class Tiger extends Animal {
32|     public String news = "Tiger's news";
33|     public String message = "Tiger's message";
34|
35|     public static String smile() {
36|         return "smile from Tiger";
37|     }
38|
39|     @Override
40|     public String getNews() {
41|         return news;
42|     }
43| }
```

ผลลัพธ์ที่จะเกิดขึ้นจากการใช้โปรแกรมต่อไปนี้คืออะไร และอธิบายเหตุผลประกอบคำตอบ

`1. x.news is Animal's news`	ค่าของตัวแปรจะมาจาก reference type
`2. ((Tiger)x).news is Tiger's news`	แปลงเป็น Tiger ทำให้ใช้ค่าแบบ Tiger
`3. x.smile() is smile from Animal`	static method ตาม reference type
`4. ((Tiger)x).smile() is smile from Tiger`	แปลงเป็น Tiger เลยใช้ static method แบบ Tiger
`5. x.getNews() is Tiger's news`	getNews() โดน override ในคลาส Tiger
`6. x.getMessage() is Animal's message`	ใช้ method ที่ได้ inherit มา โดยไม่ override

5.กำหนดความสัมพันธ์ของคลาส Employee และ Manager ในรูปของไดอะแกรม ดังนี้



ให้เขียนโปรแกรมเพื่อจำลองการทำงานของโครงสร้างของคลาสดังกล่าว และประยุกต์ใช้ Polymorphism สำหรับการสร้าง วัตถุชนิด `Manager` โดยกำหนดให้สร้างวัตถุ ในคลาส Main ด้วยคำสั่ง `Employee emp = new Manager();` โดยให้กำหนดที่อยู่ เงินเดือนและรายละเอียดของพนักงานได้พร้อมทั้งสามารถพิมพ์รายละเอียดของวัตถุดังกล่าวได้

Output จาก P05_EmployeeManager.java:

```
Employee ID: 1001
Name: John Doe
Salary: 12345.67
Address: 123 Somewhere Street, Someplace
Parking No: P-101
```